

RRSS	
Configuration and Change Management Report	Date: 2024.05.23

RRSS

Configuration and Change Management Report

1 Introduction

This document entails the procedures for product modifications that our group will be adopting during the development process.

2 Purpose

Configuration and change management procedures are essential as we're aiming to have a certain degree of quality for our product which is not easy to do while having multiple developers. This will ensure that we will be able to work together efficiently, improve code quality and decrease the probability of introducing bugs to the system.

3 Configuration and Change Management Specifications

- We will be using Git as our version control software.
- We will be using GitHub for the distribution of the code. (as mandated.)
- For each non-trivial change request and a bug report, an issue will be opened through the GitHub UI. The development team will discuss the best way to proceed if necessary.
- Each developer will be using separate branches for each feature and bug fix.
- The work will be done in atomic-sized commits, where a single commit will only contain relevant changes.
- The commits will have a commit message in the form of {modules}: {short change description}. The modules will be a comma separated list of modules in which changes were made. The short change description will describe what changes were done in a single sentence. If changes do not fit in a single sentence then that means that commit should be split apart.
- If there are non-trivial changes in a commit, a longer commit description that entails what and why was changed will be included in the commit.
- When the changes are done a pull request to the main branch will be opened. The pull request will be linked with the relevant issue if present.
- When the pull-request is opened, an automated CI runner(a.k.a. GitHub Actions) will run lint, formatting and automated integration tests against the changed code. If the pull-request does not pass the tests, the system will block the pull-request from being merged.
- After that, developers responsible for the changed modules will be asked to submit a review on the code. The developers will be mentioned in the CODEOWNERS file. After the code has been approved, the pull-request will be merged.
- When a sufficient set of changes are done, the project will move to a release stage. The code from main will be pushed to the staging branch for testing release-candidates. All feature development will cease during the release stage and only bug fixes will be allowed.
- When the code is sufficiently tested, the HEAD of the staging branch will be tagged as the release. A subsequent release will be made on the GitHub UI following the release tag.
- When the release on GitHub is made, an automated system will be triggered(using the GitHub Actions) to compile and add release artifacts to the release.

4 Key Considerations

- It's assumed that each developer in the group has at least a basic understanding of Git and GitHub.
 - Knowing more advanced git features like git bisect is recommended for more efficient development.
- The automated CI checks will be available as a GitHub Action Workflow.
 - Waiting for CI results might be suboptimal for developer experience. A method to run the checks locally will be provided.